

SmuFuzz: Enable Deep System Management Mode Fuzzing in Fully Featured UEFI Runtime Environment

Jianqiang Wang
CISPA
jianqiang.wang@cispa.de

Yi Xiang
Zhejiang University and EPFL
xiangyi0406@zju.edu.cn

Meng Wang
CISPA
meng.wang@cispa.de

Qinying Wang
EPFL and Zhejiang University
qinying.wang@epfl.ch

Ali Abbasi
CISPA
abbasi@cispa.de

Thorsten Holz
Max Planck Institute for Security & Privacy
thorsten.holz@mpi-sp.org

Abstract—As part of the UEFI standard, *System Management Mode* (SMM) was introduced on x86 processors to handle critical hardware events. With strict access control to this operating mode, SMM applications run at a high privilege level (known as *Ring -2*), in which they have (almost) unlimited access to system resources. However, vendors commonly use memory-unsafe system programming languages to develop SMM applications, which makes them vulnerable to memory corruption and an appealing target for attackers. Fuzzing is an effective method for detecting memory corruption vulnerabilities across a wide range of applications. Unfortunately, existing approaches for testing SMM applications lack a UEFI runtime environment to properly support SMM application execution. Without this environment, application data is often not correctly initialized. Once such uninitialized data is accessed during fuzzing, it causes premature exits or unintentional crashes. As a result, existing methods can only explore *shallow* parts and often produce *high false-positive* rates.

In this paper, we propose SmuFuzz, a fuzzing framework designed to detect vulnerabilities in closed-source SMM applications distributed by vendors. SmuFuzz overcomes prior limitations by partially rehosting SMM applications within a custom infrastructure that provides a fully featured UEFI runtime environment. This infrastructure provides the necessary dependencies and runtime for SMM application preparation, initialization, and finalization. In addition, SmuFuzz automatically infers the complex SMM application input semantics for deep exploration. In our experiment, SmuFuzz achieved 4.45x higher unique basic block coverage compared to state-of-the-art fuzzers. It also found more vulnerabilities while significantly reducing false positives. Using SmuFuzz, we identified 38 new vulnerabilities in firmware from major vendors, all of which were disclosed responsibly.

1. Introduction

Modern computers rely on processor privilege levels to isolate different execution environments and ensure system security. In this model, higher-privileged software provides

interfaces to lower-privileged software while performing necessary security checks to guard against malicious requests. Similar to kernel mode (*Ring 0*), *System Management Mode* (SMM) is an even higher privilege (*Ring -2*) operating mode provided by the x86 processor. Introduced in the 1990s [31], SMM is designed to handle critical hardware events such as power management and chipset errors. The processor implements SMM by enforcing access control through a physical memory region called *System Management RAM* (SMRAM). All read and write accesses from outside of SMM, even with *Ring 0* privilege, are rejected. The processor enters SMM by signifying a *System Management Interrupt* (SMI). SMIs can be explicitly triggered by writing to a designated I/O port for user-specified requests or automatically triggered by the processor in response to asynchronous hardware events. The SMI handlers function similarly to system calls, which validate incoming requests and perform the appropriate privileged tasks.

Today, vendors increasingly extend SMM applications to accomplish security-critical functionalities, such as managing the Trusted Platform Module (TPM). As a result, these applications now expose more interfaces than before. Unfortunately, memory-unsafe programming languages such as C and C++ are widely used in SMM development, making them prone to memory corruption vulnerabilities. Even worse, the SMM application is physically located in flash memory, allowing malicious changes to persist even after an OS reinstall. As a result, the high privilege level of SMM and the system call-like interfaces make SMM applications attractive targets for attackers seeking persistent access to systems. As reported by Lenovo [26], a memory corruption vulnerability, which gives the attacker arbitrary code execution in SMM, affected a wide range of Lenovo PCs, workstations, and laptops. Another two vulnerabilities, [CVE-2022-27493](#) and [CVE-2022-40261](#), reported by Binary [5], could be exploited to escalate privilege from *Ring 0* to *Ring -2* on Intel platforms. Binary also noted that attackers could exploit the vulnerability to install a backdoor in flash memory, which can survive a hard drive replacement, thereby giving the attacker permanent control.

Static and dynamic methods have been proposed to detect memory corruption vulnerabilities in SMM applications. For static methods, efiXplorer [1], [2] and SPENDER [48] are two methods that aim to identify SMM application vulnerabilities. However, they fall short in several aspects: First, they only focus on specific types of vulnerabilities, such as *callout*, a defect that causes an unintentional branch from SMRAM to outside, leaving other types of vulnerabilities, such as buffer overflows, untested. Second, like other static analysis tools, they suffer from inaccuracies in data and control flow analysis when dealing with large and complex applications. Third, these tools cannot effectively identify the full range of attack surfaces, limiting their ability to detect vulnerabilities beyond predefined patterns.

For dynamic methods, security assessment frameworks, such as CHIPSEC [23], lack access to SMM internal states. Thus, they must treat the SMM application as a black box, making it difficult to reach or analyze complex internal logic. *Fuzzing* has proven to be an effective dynamic analysis method for vulnerability detection [16], [30], [38]. However, when targeting SMM applications, several problems still hinder both deep code exploration and precise vulnerability detection. To benefit from modern fuzzing mechanisms, such as coverage feedback and snapshotting, rehosting is the most effective approach [11], [35]. However, rehosting the full UEFI firmware, which includes SMM applications, presents significant challenges. The firmware requires access to thousands of processor and peripheral registers. These accesses involve complex interaction logic, most of which is not standardized or documented. None of the existing simulators can initially support the full firmware simulation for any platform. The state-of-the-art UEFI firmware fuzzing frameworks, proposed by Yang et al. [46] and Hart et al. [34], use Simics [21] as the virtual platform simulator for firmware fuzzing. They both use Simics to boot the firmware and manually specify the entry point to inject fuzz input. When targeting SMM applications designed for a wide range of devices, Simics only offers limited platform simulations [21], which means the majority of the firmware cannot boot from the simulator. Existing works did not propose any method to tackle this problem.

While rehosting the firmware is already challenging, the runtime environment to support SMM application execution is also essential, but even harder to provide. SMM applications rely on the runtime to set up the execution environment, resolve dependencies, initialize the SMM context, finalize SMM protection, prepare the SMI context, and dispatch SMI requests. Without such an environment, the applications may not be initialized correctly, and SMI requests may not be handled correctly. Once the incorrectly initialized data is accessed during fuzzing, it causes early exits or unintentional crashes. None of the existing firmware development frameworks is designed to load and initialize any SMM applications. Directly applying the development frameworks to an SMM application may result in infinite loops or early crashes. Developing a universal runtime environment from scratch that is compatible with any SMM applications requires massive manual efforts. Programmable

simulator Qiling [33] and RSFUZZER [47] provide a customized loader for raw firmware executables. However, the loader is not compatible with arbitrary SMM applications and is far from sufficient to create a runtime environment that supports the execution of SMM applications.

Even with a fully featured runtime environment, deep code exploration of SMM application fuzzing is still hindered by the complex SMI handler semantics. Different handlers may also interfere with each other’s control flow. Vendors do not release the source code or the handler specifications, which causes manual reverse engineering effort to infer the intra- and inter-handler semantics.

We present key insights and practical solutions to deal with the above three fuzzing challenges. First, we decouple the firmware rehosting problem into a partial SMM application rehosting problem. The key observation is that, although the SMM application is embedded in the firmware, it does not directly interact with the hardware but instead mainly relies on *protocols*. This is a communication method used within UEFI firmware, which will be explained in Section 2.2. We follow the least context principle to run the SMM applications by providing the necessary protocol instances. In addition to necessary protocols, we adapt an existing firmware development framework to offer a fully featured UEFI runtime environment for SMM application preparation, initialization, and finalization. Moreover, to deeply fuzz SMI handlers, we intercept memory access to infer the intra-handler nested structure semantics and group different SMI handlers based on their dependency relationships to infer the inter-handler interference semantics.

Based on these insights, we propose SmuFuzz, a fuzzing framework designed to detect memory corruption vulnerabilities in closed-source vendor SMM applications. SmuFuzz offers a fuzzing-retrofitted infrastructure. The infrastructure is compatible with the underlying simulator and can directly boot from it. SmuFuzz splits the SMM application into multiple SMM modules. The infrastructure provides necessary protocol dependencies and a fully featured UEFI runtime environment for running the modules. The infrastructure also provides a fuzzing harness, which is responsible for interacting with the fuzzer, grouping handlers, triggering SMIs, supplying fuzz input, and capturing the crashes. From a high-level view, SmuFuzz consists of three main phases: In the first phase, we extract SMM modules from the vendor UEFI firmware and then embed the modules into our infrastructure firmware. In the second phase, we boot the infrastructure firmware in the simulator and initialize the SMM modules. In the third phase, after all modules are initialized, the harness starts fuzzing by explicitly triggering SMIs and feeding fuzz input to the handlers. We evaluated SmuFuzz on 31 UEFI firmware from nine brands. In our experiments, SmuFuzz outperformed the state-of-the-art SMM application fuzzing tool RSFUZZER and the leading static analysis tool efiXplorer in both coverage and vulnerability detection.

Contributions. We make the following key contributions:

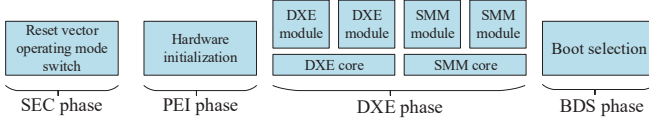


Figure 1. Overview of four phases of UEFI boot.

- We identified several obstacles that hinder SMM application fuzzing: difficulties of firmware rehosting, the lack of a fully featured UEFI runtime environment, and the complexity of SMI handler semantics.
- We present the design and implementation of SmuFuzz to test SMM applications. SmuFuzz offers a fuzzing-retrofitted infrastructure to rehost SMM applications. It provides necessary protocol dependencies and a fully featured UEFI runtime environment, as well as a universal fuzzing harness to deeply fuzz SMI handlers. We open-source SmuFuzz at [GitHub](#).
- We evaluated SmuFuzz on 31 UEFI firmware. It achieved 4.45x greater unique basic block coverage, found more vulnerabilities, and caused fewer false positives compared to state-of-the-art works. In addition, we discovered 38 new vulnerabilities in these firmware samples, all of which were disclosed responsibly.

2. Technical Background

An SMM application is a component of the UEFI firmware and starts execution along with it. This section begins by outlining the four phases of the UEFI boot process and the modular design of the UEFI standard. Next, we present the layout of the UEFI firmware and demonstrate methods for extracting and embedding closed-source vendor SMM modules. Finally, we introduce the SMM life cycle, detailing the process from module initialization to finalization and SMI request handling.

2.1. UEFI Boot Flow

A UEFI firmware is stored in flash memory and is automatically memory-mapped into the system address space upon power-on. The processor begins execution from a fixed reset vector, marking the entry point into the first phase of the UEFI boot process: the Security (SEC) phase. As shown in Figure 1, the UEFI boot flow comprises four phases: Security (SEC), Pre-EFI Initialization (PEI), Driver eXecution Environment (DXE), and Boot Device Selection (BDS), as defined in the UEFI specification [42]. Below, we briefly describe each phase and indicate where the SMM application is initialized and takes control.

The SEC phase starts at the processor’s reset vector. It handles early system initialization tasks, including reset handling, switching the operating mode, and setting up temporary memory. It then transfers control to the PEI phase. The PEI phase is responsible for low-level hardware initialization, such as CPU configuration and DRAM setup.

```

EFI_STATUS EFI_IMAGE_ENTRY_POINT(
    IN EFI_HANDLE ImageHandle,
    IN EFI_SYSTEM_TABLE *SystemTable
);

```

Listing 1: Module initialization function prototype.

```

// Common Header.h
struct ExampleProtocol {
    // any user-specified fields
};
struct GUID ProtocolGuid = {xxxx-xxxx-xxxx-xxxx};
// Module A.c
struct ExampleProtocol ProtocolProduceInstance;
InstallProtocol(ProtocolGuid, &ProtocolProduceInstance);
// Module B.c
struct ExampleProtocol *ProtocolConsumeInstance;
LocateProtocol(ProtocolGuid, &ProtocolConsumeInstance);

```

Listing 2: Protocol usage pseudo code.

Once complete, control is handed over to the DXE phase. The DXE phase is the most complex one. It loads and executes various DXE modules that initialize a wide range of platform components, such as chipset features, I/O buses, and file systems. Crucially, the SMM application is also initialized during this phase. When the SMM-related files are detected, the firmware loads them into memory, initializes the SMM execution environment, and activates SMM services. In the final BDS phase, the firmware enumerates bootable devices and selects one according to platform policy or user configuration. Control is then transferred to the OS loader. Unlike legacy BIOS, UEFI firmware remains resident in memory after boot to provide runtime services, including access to SMM services, to the operating system.

2.2. UEFI Modular Design

UEFI adopts a modular architecture design. As illustrated in Figure 1, during the PEI and DXE phases, a core is responsible for initializing the execution environment and managing various modules. (Note: The design of the PEI phase is omitted in the figure, as our focus is on SMM.) The core responds to the newly discovered modules, loads modules into memory, and then invokes their initialization functions to set up the modules’ execution context. All module initialization functions follow a standardized function prototype, as shown in Listing 1. The second argument, `SystemTable`, is a pointer to a service table that provides functionalities such as memory allocation, I/O access, and protocol management. Modules leverage this service table to configure their execution environment and access platform resources. The return type, `EFI_STATUS`, indicates the success or failure of the initialization. A return value of `EFI_SUCCESS` (typically defined as integer 0) signifies successful initialization; any other value indicates an error. Communication between modules is handled through *Protocols*. Protocol dependencies determine the load order and ensure that modules are initialized in the correct sequence.

Protocol. In UEFI, a *protocol* refers to an interface produced and consumed by the core and its modules. At a low level, a protocol is simply a binding between

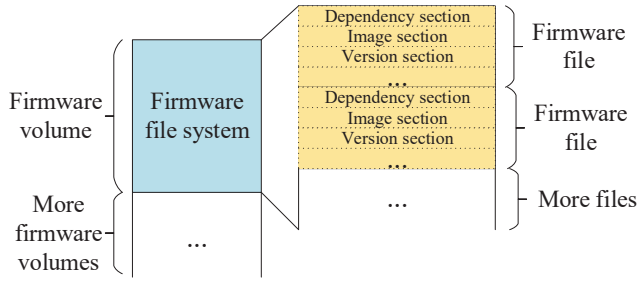


Figure 2. UEFI firmware storage layout.

a pointer and a globally unique identifier (GUID) [45]. As shown in Listing 2, a protocol is produced in module A via the `InstallProtocol` service provided by `SystemTable`. The protocol binds a protocol instance (i.e., the pointer `&ProtocolProduceInstance`) with the corresponding GUID `ProtocolGuid`. A hash table is created during core initialization to record this mapping. When another module (e.g., module B) attempts to locate the protocol using the same GUID, the core searches the hash table and returns the associated pointer. Note that we will use the terms *locate/consume* and *register/produce* interchangeably in this paper. Unless explicitly handled by developers, the producer and consumer of a protocol must agree on the structure layout and semantics of the protocol data; otherwise, the consumer module cannot correctly interpret the pointer returned by `LocateProtocol`. SMM applications also utilize a set of SMM-specific protocols. However, these protocols must be produced, consumed, and used exclusively within SMRAM. We refer to protocols produced and consumed by standard DXE modules as *DXE protocols* and those used within SMRAM as *SMM protocols*.

Module Dependency. A module may declare dependencies on specific protocols that must be available before it can be loaded. When a module is identified in the firmware, it is not loaded immediately. Instead, the core first inspects the module’s protocol dependencies. A module is only loaded and initialized once all of its required protocols have been registered. Since protocols can be produced during the initialization of modules, dependency resolution becomes a recursive process. The core begins by loading modules with no dependencies. Each time a module is successfully initialized and registers one or more protocols, the core re-evaluates the list of remaining modules to identify any whose dependencies are now satisfied. This iterative process continues until no further modules can be loaded, ensuring that all dependency chains are respected.

2.3. UEFI Firmware Layout

The UEFI firmware layout can be nested and complex. We provide a simplified description, as shown in Figure 2, which suffices for understanding the key concepts. UEFI files are stored in firmware volumes, which are formatted as *Firmware File Systems* (FFS). It is essential to note that a

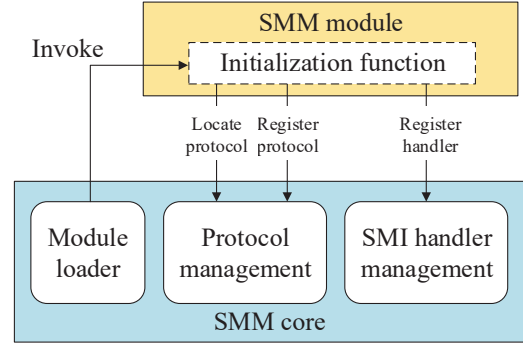


Figure 3. SMM initialization stage.

firmware file is not a conventional file, but rather a combination of multiple sections. This paper focuses on two specific sections: the dependency section and the image section. The dependency section contains information about the module’s dependencies, which the core uses during module loading. The image section contains the actual executable that will be loaded into memory. For simplicity, we refer to a module as a *UEFI file* rather than just the executable. Due to the well-defined FFS structure, UEFI modules can be easily extracted from and embedded into a UEFI firmware by using a standard FFS editor.

2.4. SMM Life Cycle

SMM application operates at a high privilege, with the processor enforcing strict access controls over a specific range of physical memory known as *System Management RAM* (SMRAM). Programs outside of SMRAM, including those with kernel-level privileges, are prohibited from accessing SMRAM. However, this protection is not always enforced. Before the SMM lock control registers are configured, SMRAM remains accessible from outside for initialization [22]. We divide the SMM life cycle into two distinct stages based on its lock state: the initialization stage and the SMI handling stage.

Initialization Stage. This stage happens in the DXE phase. As shown in Figure 3, during this stage, once a new SMM module is discovered in the firmware, a *Dispatch* event is delivered. The SMM core responds to the event, resolves the module dependencies, if satisfied, loads it into SMRAM, and invokes its initialization function. SMM modules may depend on both DXE and SMM protocols to complete their initialization (though we omit the DXE protocols in the figure for simplicity). In this stage, SMM modules register zero or multiple *System Management Interrupt* (SMI) handlers and SMM protocols. Similar to protocols, each SMI handler is associated with a GUID. If the GUID specified by a module is a NULL pointer, the handler is considered a *root SMI handler*. Otherwise, it is classified as a *non-root SMI handler*. After completing a module initialization, the core proceeds with the next module, utilizing the newly registered SMM protocol for

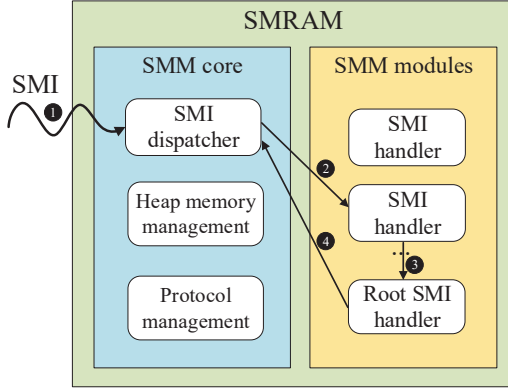


Figure 4. SMM SMI handling stage.

dependency resolution. Once all modules are initialized, or no further dependencies can be resolved, a *Lock* event is delivered. The module may register a callback function to respond to this event for context finalization. After handling the event, the SMM lock control registers are configured so that SMRAM cannot be accessed from outside. The SMM application cannot invoke non-SMRAM functions after the event; otherwise, it causes a callout vulnerability [6], [7].

SMI Handling Stage. This stage happens in the BDS phase. Following the Lock event, the only way to enter SMM is to trigger an SMI. An SMI can be triggered in two ways: Explicitly, by writing a value to I/O port 0xB2 [22]. Automatically, through periodic processor-generated SMIs. The root SMI handlers are usually used for processor-generated SMIs. As shown in Figure 4, when an SMI occurs, the processor performs a context switch into SMM, suspending the execution of all other CPU cores. The SMM core then takes control, reads the SMI request parameter, including the handler GUID, from a predefined non-SMRAM region, and then dispatches the SMI request to the registered SMI handlers. During the handler’s execution, the core provides SMM protocol and SMRAM heap memory management via a service table. After the specific handlers are invoked, the dispatcher always invokes the root handler to ensure hardware events are processed. If the GUID parameter passed in is a NULL pointer, the dispatcher directly invokes the root SMI handler. After servicing the request, the core reclaims control, restores the context, and exits SMM.

3. Key Challenges and Insights

From rehosting to deep exploration, we summarize three challenges in SMM application fuzzing and propose three insights to address them, respectively.

3.1. Key Challenges

#1: Difficulty of Firmware Rehosting. On-device fuzzing cannot benefit from modern fuzzing frameworks, such as AFL [30], AFL++ [16], and LibAFL [17], due to

```
EFI_STATUS EFI_MM_HANDLER_ENTRY_POINT(
    IN EFI_HANDLE DispatchHandle,
    IN CONST VOID *Context OPTIONAL,
    IN OUT VOID *CommBuffer OPTIONAL,
    IN OUT UINTN *CommBufferSize OPTIONAL);
```

Listing 3: SMI handler function prototype.

the firmware bare-metal environment. As such, rehosting is a very promising way. However, existing rehosting techniques, such as HALucinator [11], P2IM [14], and Hoedur [36], primarily target RTOS firmware, which is considerably less complicated compared to UEFI firmware. An UEFI firmware requires access to thousands of processor and peripheral registers, many of which are non-standardized. Complex communication logic is involved during the interaction with them. On x86 processors, both Port I/O (PIO) and Memory-Mapped I/O (MMIO) are used for peripheral access, making it even harder to infer peripheral I/O addresses. Moreover, vendors do not always release the full firmware, but only the modified part, for firmware updates, making it impossible to boot the firmware.

#2: Lack of Fully Featured UEFI Runtime Environment. A fully featured UEFI runtime environment includes several necessary components, such as module loaders, dependency resolvers, and event managers, to correctly initialize the SMM application. UEFI development frameworks, such as EDK2 [13] and GNU-EFI [24], provide a fully featured runtime environment, but they are not designed to support arbitrary SMM applications. Directly composing the framework with the vendor SMM applications leads to a dead loop or early crash during booting. Programmable simulators like Qiling [33] attempt to provide a straightforward loader for a single SMM executable, but they fall short in several critical areas:

- The simulators offer minimal support for DXE protocols, which are essential for loading SMM modules.
- The simulators can only support a single module at a time. Even with customized multi-module support (e.g., RSFuzzer [47]), module protocol dependency resolution is not considered, resulting in an inaccurate or incomplete module loading.
- The simulators do not offer a fully featured UEFI runtime environment. For example, they do not signal the Lock event after the initialization stage, and do not perform a context switch when handling an SMI. Lacking these features leads to incorrect memory and register initialization, which can result in false positives during the fuzzing process.

Implementing a generic runtime environment that is compatible with *any* SMM applications from scratch requires abundant manual efforts. Therefore, providing a fully featured UEFI runtime environment is a challenging project.

#3: Complex SMI Handler Semantics. Unlike system-call interfaces, there is no standardized specification for SMI handlers. As shown in Listing 3, all SMI handlers share the same function prototype. The second parameter, *CommBuffer*, is a *VOID* pointer that references the input data. However, the interpretation and usage of this buffer

entirely depend on the SMI handler’s implementation, leaving it to the discretion of the handler developers. In the case of system-call fuzzing, manually specifying system-call fuzzing interactions [19] (e.g., opening a file with a path string, then operating on the returned handler) is straightforward. However, vendors commonly do not release the UEFI firmware source code or handler specifications and do not always fully comply with the UEFI standard. The closed-source ecosystem makes it challenging to analyze intra- and inter-handler semantics, further hindering the fuzzing process in shallow areas.

3.2. Key Insights

#1: Partial SMM Application Rehosting. We observe that UEFI firmware follows a layered design: low-level modules perform complex I/O, whereas high-level modules, including SMM modules, mainly consume their outputs. Thus, while I/O complexity obstructs full firmware rehosting, it can be addressed by our proposed partial SMM rehosting; occasional I/O operations in SMM can be handled by fuzzing. Although UEFI firmware is designed to run on specific platforms, SMM modules primarily rely on protocols rather than direct hardware interactions. SMM modules can be initialized when their protocol dependencies are satisfied. Both DXE protocols and SMM protocols are necessary for SMM module initialization. While SMM protocols are designed to be used within SMM modules, DXE protocols are extra dependencies that SMM applications need. To confine the dependency resolving problem inside SMM, we follow the least context principle. We inspect all the SMM modules’ dependency sections and extract the DXE protocol dependencies. For the standard DXE protocols, we implement standardized protocol instances to provide accurate SMM information. For the proprietary protocols, we found that developers tend to include solely two types of data fields: integer and function pointer. We synthesize their instances with fuzz values and dummy function pointers.

#2: Adaptive UEFI Runtime Environment. While implementing a runtime from scratch requires abundant manual effort, adapting an existing UEFI development framework to support arbitrary SMM applications is acceptable. We observe that the compatibility problem between the UEFI framework and the SMM application arises from a mismatch in system configuration and information. The SMM modules require specific data, such as I/O register values and NVRAM variables, to complete the initialization process. Once the data is not valid or the accessed address is not pre-allocated, a loop or crash may occur. We provide the requested data for the modules and explore the paths in the module initialization function. A correct code returned by the initialization function indicates a successful execution. We utilize fuzzing to fulfill the requested data and apply it to each module initialization function. As such, SMM modules can be correctly initialized one by one.

#3: Automate SMI Handler Semantics Inference. After successfully initializing the SMM application, SMI handler fuzzing is hindered in shallow exploration if the

SMI handler semantics are unknown. To infer the intra-handler semantics, we must address the nested structure inference problem [8], [12] (i.e., pointer analysis). We use an interceptor to solve this problem. When an accessed memory address is not in the range of SMRAM during handler execution, we redirect the access to a fixed location from which the handler gets a fuzz value. Once a data field is interpreted as a pointer, it likely fetches a fuzz value instead of reading a fixed data or causing a crash. As such, the nested structure problem is automatically solved by the interceptor mechanism. To infer the inter-handler semantics, we record the SMI handler, protocol registration, and consumption information during the SMM module initialization function execution. The information is used to analyze the handler’s dependencies. We consider two types of handler dependencies: cross-module dependency and single-module dependency. Handlers registered in the same module may share global variables, enabling one handler to modify the global variables to influence the execution of another. Handlers registered in different modules can communicate and influence each other’s execution through protocols. We categorize the SMI handlers with dependency relationships into a group and fuzz a group at a time. In this way, even without the concrete semantics of SMI handlers, we still maximize the possibility of solving the inter-handler obstacles by using existing fuzz strategies such as Cmplog [3].

We design a fuzzing-retrofitted infrastructure as an all-in-one solution to implement the insights described above. This infrastructure can be compiled into an UEFI firmware and run seamlessly in the simulator without any compatibility problems from the SEC phase to the BDS phase. Specifically, we benefit from the following infrastructure advantages:

- It provides a fully featured UEFI runtime environment, including rich DXE protocol instances, SMM module dependency resolution, multi-module loading support, and various SMM life cycle events, etc.
- The infrastructure manages the SMM modules and communicates with the fuzzer. It provides fine-grained execution control over the modules and utilizes fuzzing to initialize them, thereby maximizing the number of loadable SMM modules.
- The infrastructure includes a universal harness to group SMI handlers, trigger SMIs, supply inputs for SMI handler fuzzing, and capture the crashes caused by invalid branch or memory accesses to detect vulnerabilities.

4. Design and Implementation

We now outline the design of SmuFuzz, which is based on our threat model for SMM memory safety. The SmuFuzz design is structured into three phases, we provide a high-level overview in Figure 5. In the composing phase, we embed the vendors’ SMM modules into the infrastructure firmware. In the initialization phase, we utilize the infrastructure to load and initialize as many SMM modules as possible. After the SMM core responds to the Lock event, we reach the final fuzzing phase, during which we perform

the actual SMI handler fuzzing. We provide detailed explanations of each phase in the following subsections.

4.1. Threat Model

Aligned with prior SMM analysis works [47], [48], we assume that the UEFI firmware remains intact and is verified during boot; thus, attackers cannot tamper with the firmware to modify code or data. During the initialization phase, the SMRAM is accessible from outside. We assume that SMM modules can be correctly initialized in their native execution environment. Therefore, crashes during this phase are not considered vulnerabilities. However, in the fuzzing phase, SMRAM is protected by the enforced access control. We assume the attacker gains complete control over non-SMRAM with Ring 0 privilege, as well as the ability to manipulate peripheral and NVRAM variables. Additionally, the attacker can explicitly trigger SMI at any time and modify the processor register values before entering SMM.

4.2. Composing Phase

As shown in Figure 5 (I: *Composing Phase*), we analyze the vendor UEFI firmware and extract modules tagged as SMM Core or SMM Module. These extracted SMM modules are then embedded into our fuzzing-retrofitted infrastructure firmware. However, we exclude the `PiSmmCpuDxeSmm` module, as it is responsible for handling interrupts and exceptions. We implement a customized `PiSmmCpuDxeSmm` module to catch crashes. We offer the following fuzzing-retrofitted capabilities in our infrastructure to facilitate SMI handler fuzzing.

Notification Stub. To accurately track and control firmware execution, we insert notification stubs that alert the fuzzer when reaching specific points. In particular, we notify the fuzzer when a module’s initialization function is about to execute or exit, and the Lock event has been handled. Along with these notifications, we report relevant parameters to the fuzzer, such as the module’s GUID, start address, and end address. Upon receiving the notification, the fuzzer pauses execution and performs necessary tasks, which will be introduced in Section 4.3.

Information Recording. The SMM core provides essential services, such as protocol and SMI handler management, through a service table. SMM modules obtain a pointer to this table and dereference the function pointers to access these services. We hook the function pointers to perform tracing tasks for information recording during the module initialization. The information includes: (i) Module metadata (GUID, start address, and end address), (ii) Protocol interactions (protocol production and consumption), and (iii) SMI handlers (GUID and module associations for installed handlers). This information enables the SMI handler semantics inference and debugging analysis. Note that root handlers are registered via several standard SMM protocols instead of the service table. We hook the protocols to get the root handler information and treat them equally in the recording.

SMI Handler Fuzzing Harness. To facilitate SMI handler fuzzing, we implement a dedicated harness that interacts with the fuzzer, groups SMI handlers, explicitly triggers SMI, and supplies fuzz inputs during testing. The details of this process are presented in Section 4.4

4.3. Initialization Phase

In this phase, we primarily leverage fuzzing to find valid paths that complete the initialization of SMM modules. Since SMI handlers can only be fuzzed if registered during the module initialization process, this phase focuses on loading and initializing as many SMM modules as possible.

Workflow. As illustrated in Figure 5 (II: *Initialization Phase*), the workflow proceeds as follows: ① The simulator runs the infrastructure firmware until an SMM module is about to be initialized by SMM core’s loader. Before executing the module’s initialization function, our notification stub is triggered. ②-③ A snapshot is taken and passed to the fuzzing engine as input. ④ The fuzzer begins testing the initialization function. ⑤-⑥ Upon completion of the initialization function, the notification stub reports the exit and its corresponding reason to the result checker. ⑦ If the checker reports a failing result, the snapshot is restored and fuzzing continues. ⑧ If a valid execution path leading to successful initialization is discovered or a timeout is reached, fuzzing is immediately halted, the snapshot is discarded, and control is returned to the SMM core. After processing one module, the SMM core reevaluates the module dependencies and tries to load the next available module. The initialization function fuzzing is repeated until no more modules can be initialized.

Fuzz Inputs. We identified several inputs that influence the control flow of the initialization function. Instead of feeding fixed values to the inputs, we provide fuzzing-generated data to explore valid execution paths. Specifically, we target the following inputs:

- Certain proprietary DXE protocols serve to initialize the module environment; our approach involves feeding fuzz inputs into their data fields.
- *Hand Off Blocks* (HOBs), generated during the PEI phase and passed to the DXE phase, provide system information essential for initializing specific SMM modules. We overwrite the HOB entry data with fuzz inputs but preserve the standard header to bypass some HOB checks. Before invoking the initialization function, we install the HOB as a configuration table from which the module gets HOB information.
- Stored in the source code and compiled into the final firmware, configuration settings allow developers to enable or disable specific features. These settings are represented as variable-length integers. We replace them with fuzz data to explore alternative configurations.
- Modules use Model-Specific Registers (MSRs) and Port I/O (PIO) Registers to determine processor or peripheral status. However, the simulator only provides platform-specific values, which may not meet expected conditions. We substitute this access with fuzz data to simulate diverse hardware states.

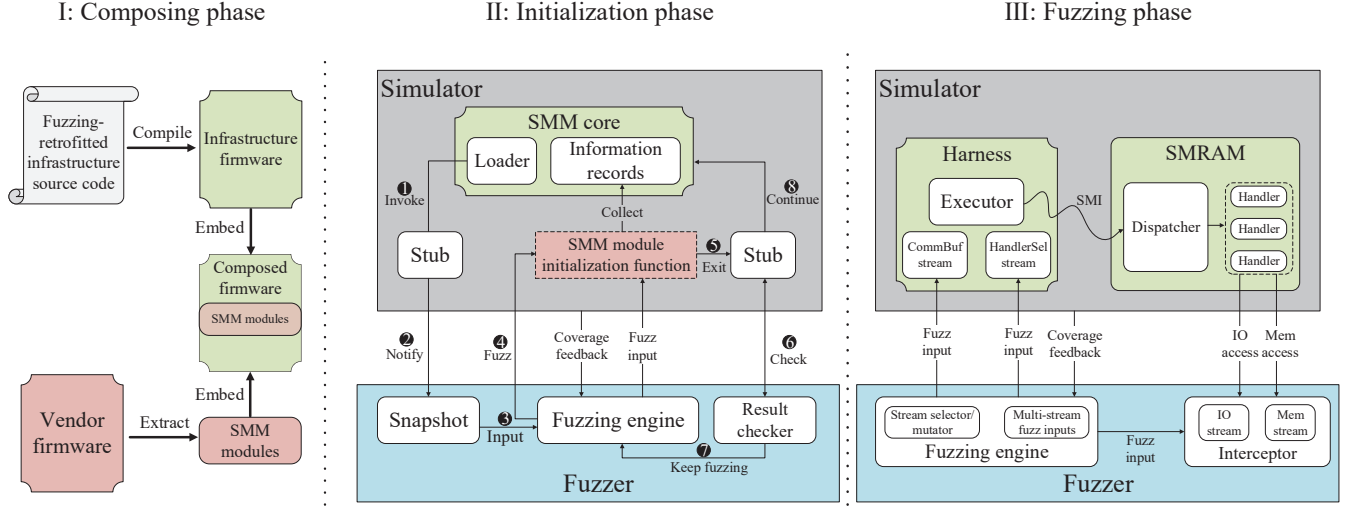


Figure 5. High-level overview of the design of SmuFuzz.

- SMM modules sometimes check NVRAM variables. We replace NVRAM values with fuzz-generated data to introduce variability.

DXE Protocol Provision. DXE protocols are important dependencies for SMM modules. These protocols provide essential information about the system, including the address and size of the SMRAM. To meet this requirement, we provide the DXE protocols necessary to initiate the SMM modules. We provide standardized DXE protocol implementation for the standard protocols. However, the vendor SMM modules commonly rely on proprietary DXE protocols whose source code is unavailable. It requires considerable manual effort to reverse engineer the details of the DXE protocol, which is not an optimal approach. Instead, we synthesize the proprietary DXE protocol by combining fuzz data and dummy functions: Regardless of size, all integer fields are filled with fuzz data. Function pointer fields are assigned pointers to dummy functions that perform no operations and always return successfully.

4.4. Fuzzing Phase

After handling the Lock event, SmuFuzz enters the final fuzzing phase. This phase focuses on exploring the deep code logic of SMI handlers and detecting memory corruption vulnerabilities. To achieve this, we integrate a fuzzing harness into the infrastructure. As shown in Figure 5 (III: Fuzzing Phase), the harness interacts with the fuzzer and explicitly triggers SMI for fuzzing.

Workflow. Each fuzzing run can be viewed as the following steps: 1) Handler selection: The harness requests fuzz input to determine which handler to fuzz. 2) Comm-Buffer preparation: The harness allocates a memory buffer and requests the fuzzer to populate it with fuzz-generated data. 3) SMI invocation: The harness explicitly triggers an SMI, prompting the infrastructure to switch context and

allowing the SMM core to dispatch the SMI. 4) Access interception: During execution, the fuzzer monitors memory and I/O access by SMI handlers. The interceptor fetches a fuzz value for this access. 5) Execution exit: The fuzzer collects execution results, identifying crashes and normal exits. It then either discards the inputs, reports a vulnerability, or saves the input to the corpus.

Multi-Stream Input. Inspired by Hoedur [36] and MultiFuzz [10], we designed a multi-stream input mechanism to mitigate the input avalanche problem. Instead of treating the fuzz input as a flat byte array, we divide it into multiple independent streams, each selected and mutated separately. Our design maintains four distinct streams: CommBuf stream, HandlerSel stream, IO stream, and Mem stream. CommBuf and Handler streams are used for populating CommBuffer and SMI handler selection, respectively. IO stream is used when handlers read from an I/O port. Mem stream is used when the handlers read from a data pointer that points to non-SMRAM. For simplicity, we treat each stream as a byte array and apply an AFL-like byte array mutator on it.

SMI Handler Grouping. While fuzzing a single SMI handler at a time limits the exploration space, randomly selecting handlers to fuzz slows down the fuzzing performance. To optimize SMI handler fuzzing, we classify handlers based on their intra-module and inter-module dependency relationships. If handlers are related, we group them together, and a group is selected at a time during fuzzing. The dependency information is recorded during the initialization phase. The harness uses the information to analyze its dependency relationships. We use a fuzz value as an index to determine which handler in the group to select when triggering an SMI.

Interceptor and Nested Structure Analysis.

We assume that the attackers can manipulate the peripheral. Thus, we intercept the I/O access and provide fuzz data when the SMI handler reads from them. Besides, to automatically infer the nested structure of the CommBuffer

input, we intercept the memory access. Instead of identifying the pointer field in the inputs, our interceptor automatically handles nested structures (i.e., when handlers access pointers in attacker-controlled inputs, such as `CommBuffer`, they retrieve fuzz values instead of fixed memory data). We assume that a field in the fuzz input translated as a pointer will most likely reference an address outside SMRAM. Thus, we intercept all non-SMRAM accesses and supply fuzzed data, ensuring that nested structures are naturally populated with fuzzed inputs. The assumption holds because SMRAM occupies only a negligible fraction of the 64-bit address space (32 MB vs. 16 EB in our simulator), making the probability of a fuzzed value falling within SMRAM about 1.8×10^{-12} ; thus, such values almost always result in non-SMRAM accesses. Specifically, it functions as follows: When a memory access occurs, we inspect the address being accessed. If it is located in SMRAM, we perform the access at the original address. Otherwise, we redirect the access to a fixed address from which the handler gets a fuzz value when it is a read operation, or we discard it when it is a write operation. If a field in the `CommBuffer` is interpreted as a pointer and is dereferenced, it likely accesses non-SMRAM, resulting in reading fuzz data. As such, the nested structure input is automatically recovered.

Vulnerability Detection and Sanitizers. We treat crashes as oracles for vulnerability detection. In `SmuFuzz`, crashes directly result from division by zero (simulator-enforced), SMM callouts (simulator-enforced), and invalid memory access. Vulnerabilities such as double-fetch and double-free faults are detected when they trigger crashes. To confirm that a crash is a vulnerability, we perform backwards execution analysis to check if it originates from attack-controlled inputs. Non-crash bugs require sanitizers, but the lack of source code limits their deployment. `SmuFuzz` currently supports a heap sanitizer to detect out-of-bound heap buffer accesses. We built an SMRAM heap sanitizer in the infrastructure by hooking the heap management service function pointer. When out-of-bound memory access happens, the processor raises an exception. `SmuFuzz` captures and reports crashes by implementing the exception handler in the customized `PiSmmCpuDxeSmm` module. Besides out-of-bound heap buffer accesses, `SmuFuzz` can also detect other types of vulnerabilities. It detects SMI call-outs since it directly triggers crashes as enforced by the simulator. Unchecked access arises when pointers are dereferenced without validation (e.g., pointing into SMRAM). To detect this, the trick is to allocate a red zone in SMRAM and insert its header address into the fuzzing dictionary (tokens used for input mutation). When the address gets injected in the fuzz inputs and dereferenced, a crash occurs, and the vulnerability is detected. Double-fetch vulnerabilities occur when the same attacker-controlled memory is accessed multiple times. The first fetch may return a benign value that passes checks, while the second yields a malicious value that causes a crash. `SmuFuzz` intercepts every non-SMRAM access and supplies fuzzed data, so two fetches of the same memory yield different values, exposing double-fetch bugs via crash detection.

4.5. Implementation

We use `UEFIExtract` [43] and `UTK` [15] to extract and embed SMM modules. The entire composing process is implemented in a Python script of approximately 180 lines of code. We implement the infrastructure based on `OVMF` [41]. We modify approximately 5,600 lines of C code to integrate the necessary DXE protocols, instrumentation, and hooks for fuzzing-retrofitted capabilities. Our simulator is adapted from `LibAFL QEMU` [27]. In addition to basic edge coverage feedback and execution control, we implement several additional features, including MSR access hooks, Port I/O hooks, memory interception, and a timeout mechanism. These adaptations involve around 300 lines of C code. The fuzzer is built on top of `LibAFL` [17], incorporating a multi-stream input mechanism based on `LibAFL's MultiInput` feature. The fuzzer consists of approximately 3,500 lines of Rust code.

5. Evaluation

In this section, we thoroughly evaluate the performance and effectiveness of `SmuFuzz`. We aim to answer the following five research questions:

RQ1: Does `SmuFuzz` provide all the necessary capabilities for fuzzing vendor SMM applications?

RQ2: Can `SmuFuzz` explore deeper code logic of SMM applications?

RQ3: Can `SmuFuzz` detect more vulnerabilities while causing fewer false positives compared to existing works?

RQ4: Does the initialization phase fuzzing help load and initialize more SMM modules?

RQ5: How much overhead and manual effort is incurred for testing?

5.1. Experiment Setup

We conducted experiments on two 79-core Intel Xeon Gold 6230 CPUs @ 2.10GHz with 188 GB RAM servers running Ubuntu 22.04.1 LTS OS. Each fuzzing run was assigned a single physical CPU core. For each target, we performed the fuzzing five times as recommended by Klees et al. [25] and Schloegel et al. [37].

5.2. Dataset Collection

Our collected firmware dataset comprises 14 firmware from the `RSFUZZER` [47] experiment and 17 latest-version firmware downloaded from the online download centers of well-known brand vendors. This dataset comprises nine well-known brands. They cover a wide range of devices, including laptops, workstations, and tablets. The details of each firmware are listed in Table 1. We believe the collected dataset is comprehensive enough to test the fuzzing performance of `SmuFuzz`.

TABLE 1. Information of the firmware dataset.

	Vendor	Type	Size (MB)	# SMM Modules
Alienware 13 R3	Dell	Laptop	12	68
Alienware X51 R3	Dell	Laptop	6.7	66
ASUS P453UJ	ASUS	Laptop	5.9	49
ASUS UN65U	ASUS	Mini PC	17	60
ASUS A407UB	ASUS	Laptop	6	58
ASUS K509FA	ASUS	Laptop	16	54
ASUS X509	ASUS	Laptop	8	58
X570 GAMING X	Gigabyte	Laptop	16	111
Z690 GAMING X	Gigabyte	Laptop	32	65
AERO 15 OLED	Gigabyte	Laptop	32	55
X3 Plus r7	Gigabyte	Laptop	8	45
X9 DT	Gigabyte	Laptop	16	48
HP20-c000	HP	Laptop	8	62
HP Obelisk 875	HP	Desktop	8	53
HP Z2 Mini G4	HP	Workstation	25	259
HP Z440	HP	Workstation	10	114
Obelisk PC 875-0000	HP	Desktop	16	62
ThinkCentre M700	Lenovo	Desktop	8	98
ThinkStation P900	Lenovo	Desktop	8	77
ThinkStation S30	Lenovo	Desktop	8.1	8
ThinkPad X1 Fold	Lenovo	Laptop	13	89
ThinkPad X1 Tablet	Lenovo	Tablet	9.1	103
ThinkPad X1 2-in-1	Lenovo	Laptop	25	92
Aspire 3 A315-58-52W4	Acer	Laptop	21	80
Aspire R 13 R5-371T-595X	Acer	Laptop	8.7	69
Microsoft Surface Go	Microsoft	Tablet	18	74
Stealth 16 Mercedes-AMG	MSI	Laptop	32	51
Creator Z16 HX Studio	MSI	Tablet	32	51
Katana 15 B12VGK	MSI	Laptop	32	53
Blade Stealth RZ09-01963	Razer	Laptop	8	48
Blade Stealth RZ09-03102	Razer	Laptop	16	53

5.3. Fuzzing Capability Analysis (RQ1)

We measure the capabilities of fuzzing tools in three aspects: source availability, target scalability, and fuzzing efficiency/effectiveness. Target scalability reflects whether the tool can fuzz any closed-source vendor SMM applications. We propose four requirements to meet the capability: (i/ii) The tools can fuzz any closed-source SMM applications (target closed-source and SMI handler), (iii) The tool does not require the real hardware to be involved to get a memory dump or device states (hardware independence), and (iv) No further effort, such as manually specifying fuzz entry points and injecting fuzz inputs, is required to fuzz new SMM applications (universal harness). The third capability reflects whether the tool can efficiently and effectively fuzz the SMM application. We propose three distinct requirements to meet the capability: (i) The tool provides a fully featured UEFI runtime environment to initialize the SMM application correctly (runtime env provision), (ii) The fuzz inputs can be injected via different interfaces, such as I/O, CommBuffer, and nested CommBuffer data pointer (comprehensive attack vector), and (iii) The tool supports snapshotting to accelerate fuzzing speed and debug analysis (snapshot support). As shown in Table 2, compared with seven popular UEFI firmware fuzzing tools, SmuFuzz can fully support all the necessary capabilities and outperforms other tools.

5.4. Fuzzing Exploration Analysis (RQ2)

We measure the exploration depth of code logic that the fuzzing tools can reach by counting the discovered unique basic blocks. As other fuzzing tools either cannot target closed-source SMM applications or require hardware to be involved, we quantitatively compare with the state-of-the-art SMM application fuzzing tool RSFUZZER [47].

Figure 6 and Figure 7 in the Appendix plot the number of discovered basic blocks over 24 hours across five fuzzing runs, with a 95% confidence interval. Note that we filter out the basic blocks of the infrastructure and the modules that are not related to SMI handling for a fair comparison with RSFUZZER. Except for ThinkStation S30, SmuFuzz achieved an average of 4.45 times the number of basic blocks for all other firmware. We manually inspected ThinkStation S30 and found that the firmware executables were not compiled into position-independent binaries, which is not supported by SmuFuzz. We found that due to the lack of a fully featured UEFI runtime environment, RSFUZZER failed to load the necessary modules, resulting in incorrect initialization of protocols and global variables. For some SMI handlers, RSFUZZER only explored the first few basic blocks and then exited immediately. For example, the HP firmware utilizes a feature called *HP core services*, which is implemented in a standalone module. The developers did not follow the standardized method (i.e., via protocols) for module communication, but instead populated a standardized string field in the SMM service table. While RSFUZZER failed to load this module, SmuFuzz loaded as many modules as possible, completing the HP core service installation. As such, when testing the SMI handlers that use the service, RSFUZZER exited immediately, while SmuFuzz can continue executing.

5.5. Vulnerability Detection Analysis (RQ3)

We measure the vulnerability detection capability by counting the true positives and false positives. To highlight the vulnerability detection capability of SmuFuzz, besides RSFUZZER, we chose an additional static analysis tool, efiXplorer [1], [2] for comparison. efiXplorer is a leading and easy-to-use UEFI firmware analysis IDA Pro plugin. It has successfully detected many SMM callout vulnerabilities [5]. Table 3 lists the number of reported and confirmed vulnerabilities by three tools. We manually confirmed all discovered vulnerabilities. efiXplorer, RSFUZZER, and SmuFuzz reported 19, 59, and 177 true vulnerabilities with 1.13%, 10.35%, and 73.12% correctness rate, respectively. SmuFuzz outperforms existing methods in both the number of correctly detected vulnerabilities and correctness rate. The vulnerabilities have been disclosed to the vendors in a coordinated way and are awaiting confirmation and patching. We do not have the details of the vulnerabilities reported by RSFUZZER authors; therefore, we made an underestimation that the dataset used by RSFUZZER was excluded when counting the 38 newly identified vulnerabilities to ensure that the vulnerabilities were exclusively identified by SmuFuzz.

We analyzed the reasons that led to false positives and false negatives produced by the three tools.

False Positive. efiXplorer produced false positives, mainly because of the lack of value flow analysis, which is only accessible at runtime. For example, it reported many callout vulnerabilities for boot service function calls. Theoretically, when an SMI handler uses function pointers that

TABLE 2. General comparison with UEFI firmware fuzzing tools. ✓ refers to full support. ▲ refers to partial support. ✗ refers to no support.

	Open-source	Target Scalability				Fuzzing Efficiency & Effectiveness		
		Target closed-source	Target SMI handler	Hardware independence	Universal harness	Runtime env provision	Comprehensive attack vec	Snapshot support
Excite [4]	✗	✗	✓	✓	✗	✗	✗	✓
HBFA [20]	✓	✗	▲	✗	✗	✗	✗	✓
CHIPSEC [23]	✓	✓	✓	✗	✗	✗	✗	✗
SIMFUZZER [46]	✗	✗	✓	✓	✗	✗	✗	✓
tsffs [34]	✓	✗	▲	✓	✗	✗	▲	✓
FUZZUER [18]	✓	✗	✗	✓	▲	✗	✗	✓
RSFUZZER [47]	✓	✓	✓	✓	✓	▲	▲	✗
SmuFuzz	✓	✓	✓	✓	✓	✓	✓	✓

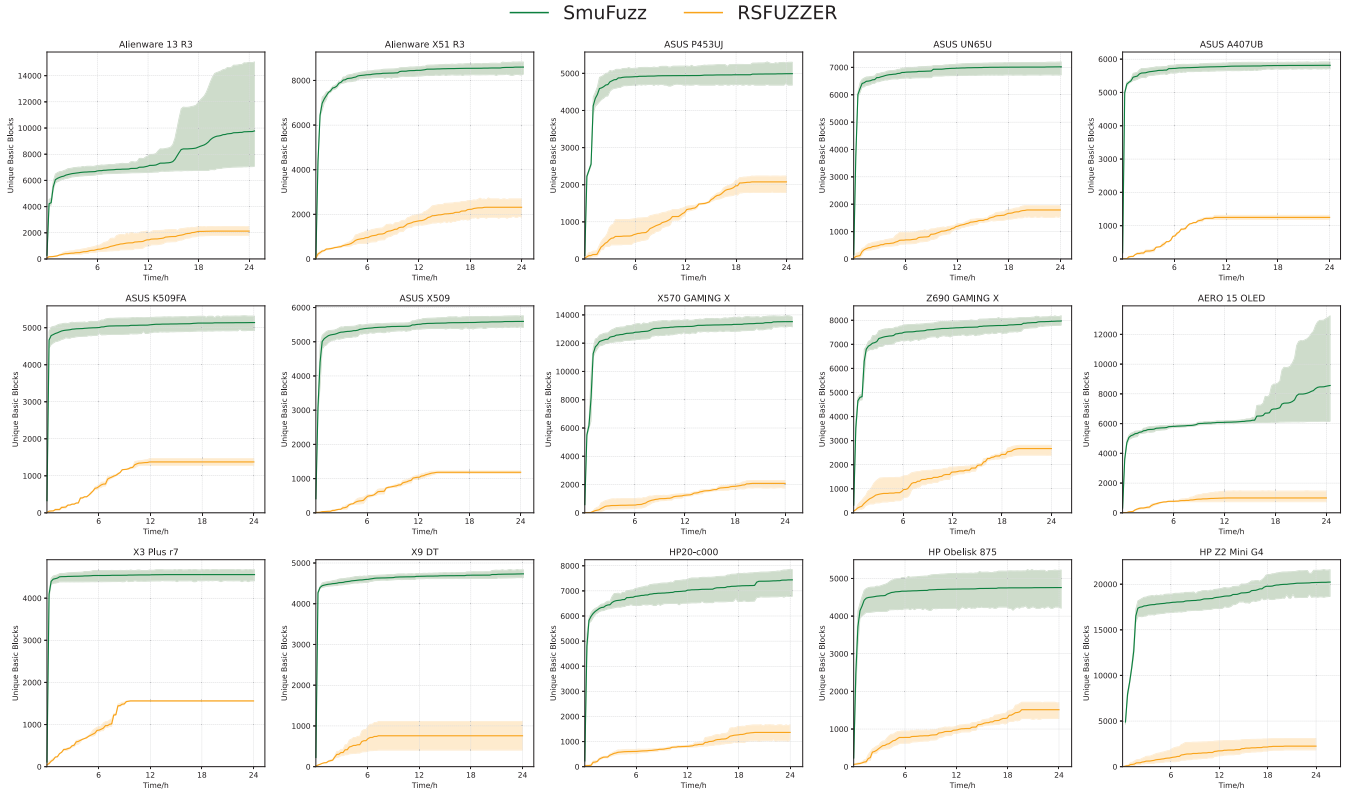


Figure 6. Basic block coverage achieved by RSFUZZER and SmuFuzz over 24 hours for five times. We plot the median and 95% confidence interval.

point to non-SMRAM to invoke the boot service function, a callout vulnerability is introduced. However, these boot service function pointers are overwritten by dummy function stubs in the SMARM during the initialization function. efiXplorer is unaware of the value assignment and treats the use of the boot service function call as an insecure behavior. However, for RSFUZZER, the false positives mainly came from the lack of a fully featured UEFI runtime environment. RSFUZZER starts from a module where SMI handlers are registered and uses force execution to execute the initialization function until the SMI handler is registered. When it reaches protocol consumption, it searches for the protocol registration module and forces the execution of the initialization function until the protocol is registered. This design

is straightforward, but it introduces two main disadvantages that can cause false positives. The first one is the inconsistent module loading order. While the SMM module dependency is complex, starting from one module initialization function and applying depth-first search on protocols does not preserve the original module loading order. Simply satisfying every protocol consumption does not reflect the original control flow. A global variable may not be correctly initialized when a module is meant to be loaded early but is loaded late. When the SMI handlers access the global variable, false positives are introduced. An example is the assignment of values to the boot service function pointers. UEFI firmware utilizes a dedicated module to establish a configuration table for function pointer overwriting. When this module is loaded

TABLE 3. Reported and true positive vulnerabilities by efiXplorer, RSFUZZER, and SmuFuzz.

	efiXplorer		RSFUZZER		SmuFuzz	
	TP	Report	TP	Report	TP	Report
Alienware 13 R3	1	69	5	27	10	10
Alienware X51 R3	1	41	9	46	12	14
ASUS P453UJ	0	45	5	22	8	8
ASUS UN65U	0	59	5	33	10	10
ASUS A407UB	0	41	0	16	4	4
ASUS K509FA	0	39	0	16	1	3
ASUS X509	1	38	0	7	3	3
X570 GAMING X	0	52	1	11	5	6
Z690 GAMING X	0	33	2	19	2	3
AERO 15 OLED	0	17	0	13	1	2
X3 Plus r7	1	21	0	19	5	6
X9 DT	1	17	0	7	2	3
HP20-c000	1	68	3	23	4	8
HP Obelisk 875	0	84	6	41	5	6
HP Z2 Mini G4	5	22	4	38	1	5
HP Z440	0	5	7	28	5	6
Obelisk PC 875-0000	0	56	0	13	1	3
ThinkCentre M700	0	38	2	18	9	11
ThinkStation P900	0	195	4	27	4	5
ThinkStation S30	0	16	2	6	0	0
ThinkPad X1 Fold	0	31	4	6	4	5
ThinkPad X1 Tablet	0	57	0	4	4	4
ThinkPad X1 2-in-1	0	11	0	5	0	3
Aspire 3 A315-58-52W4	0	97	0	7	2	4
Aspire R 13 R5-371T-595X	4	172	0	21	6	7
Microsoft Surface Go	3	161	0	14	1	8
Stealth 16 Mercedes-AMG	0	43	0	11	0	1
Creator Z16 HX Studio	0	44	0	18	0	2
Katana 15 B12VGK	0	44	0	18	0	1
Blade Stealth RZ09-01963	1	32	0	19	5	6
Blade Stealth RZ09-03102	0	31	0	17	3	3
	19	1679	59	570	117	160

later than expected, other modules cannot correctly set up the dummy boot service function. The second issue is the use of uninitialized global variables in the SMI handlers. The force execution may miss a valid path in which a global memory buffer is allocated. The SMI handler directly uses the buffer, causing a crash, assuming that the initialization function has successfully allocated the memory. Two main reasons lead to false positives for SmuFuzz. The first reason is introduced by using invalid values generated during the initialization phase. In the initialization phase, we use fuzzing to find a valid path leading to the successful execution of the initialization function. However, the fuzz-generated values that satisfy the path constraints may lead to unintended crashes in the fuzzing phase. While the fuzz-generated values do not cause crashes in the initialization phase, they cause crashes during the fuzzing phase (i.e., false positives in SMI fuzzing). A typical example is the fuzz-generated configuration data. The configuration data is fixed during firmware compilation; however, to find a valid path to reach the end of the initialization function, we use a fuzz value to explore the paths. This incurs problems when the configuration data contains a pointer value. Dereferencing the fuzz-generated pointer value in the fuzzing phase may cause unintended crashes. The other reason is the incorrect provision of the DXE protocol. Similar to the unintended crashes caused by the fuzz-generated values. Invalid provision of the DXE protocol also causes crashes in the fuzzing phase due to two types of protocol data: integer type and function pointer type. For the integer data type, the data may be interpreted as a pointer and dereferenced in the fuzzing phase. As we do not have the implementation details of the protocol, we fill the integer data field with fuzz-generated values. Once these values are dereferenced,

they may cause crashes. For the function pointer data type, we synthesize proprietary DXE protocols by using dummy function pointers. The dummy functions may miss essential data initialization, which is accessed during SMI handling. The false positive rates are 98%, 89%, and 28% for efiXplorer, RSFUZZER, and SmuFuzz, respectively.

False Negative. Regarding false negatives, since we used UEFI firmware in real devices for experiments, we do not have the exact number of vulnerabilities contained in the firmware as ground truth. Therefore, we analyze the vulnerabilities that can be exclusively detected by a single tool. efiXplorer, as an IDA Pro plugin, works on a single SMM executable. The primary reason for the false negative is the lack of inter-module analysis. As an example, an SMI handler registered in a module invokes another module’s function via a protocol. efiXplorer cannot detect a vulnerability in the other function since the control flow analysis starts from the SMI handler entry point and terminates when it reaches the function pointer’s invocation. Another reason is that efiXplorer can only detect a specific pattern of callout vulnerabilities, leaving other types of vulnerabilities undetected. For RSFUZZER, the false positives also originated from the lack of a fully featured UEFI runtime environment. For example, a complete SMI handling contains a full context switch to SMM. However, RSFUZZER fails to identify the register value that can be controlled by the attacker before the context switch as malicious input. Thus, RSFUZZER cannot detect the memory corruptions caused by a crafted attacker-controlled register value. Compared with RSFUZZER, two reasons lead to missing detections for SmuFuzz. The first reason is inadequate SMM module loading: an SMI handler can only be fuzzed when its module is loaded and the initialization function is successfully initialized. However, since the modules follow a recursive dependency relationship, an unsuccessful module initialization causes necessary SMM protocols to be uninitialized, which in turn causes the modules that depend on them to fail to load. The SMI handlers registered in such modules cannot be fuzzed and, therefore, cause missing detection. The second reason is the lack of sanitizers: SmuFuzz can only detect the vulnerabilities that cause crashes. However, not all vulnerabilities lead to direct crashes. To detect them, sanitizers are required to intentionally trigger a crash. Currently, SmuFuzz supports a heap sanitizer that can detect out-of-bound heap buffer access. Due to the lack of necessary sanitizers, vulnerabilities such as use-after-free, double free, and stack overflow cannot be detected. We do not have a real ground truth of the vendor firmware. If we consider the union of the true positives identified by the three tools as a rough ground truth, the false negative rates are 88%, 65%, and 30% for efiXplorer, RSFUZZER, and SmuFuzz, respectively.

Vulnerability Case Study. In this paper, we refrain from discussing the details of the unpatched vulnerabilities. Thanks to the SMI handler grouping algorithm, SmuFuzz detected cross-handler vulnerabilities as RSFUZZER. Besides, SmuFuzz successfully identified a new type of vulnerability, double-fetch vulnerabilities [9], [32], which

```

EFI_STATUS SMIHandlerA_1850(... __int64 *CommBuffer...) {
    qword_26F0 = *CommBuffer;
    ...
}
SMIHandlerB_1388(... __int64 *CommBuffer...) {
    ...
    SetMemory((int *)qword_26F0, 0, 0x200);
    ...
}

```

Listing 4: A cross-handler vulnerability in Dell UEFI firmware

```

EFI_STATUS SMIHandler_CD0(... __int64 *CommBuffer...) {
    ...
    VOID *v4 = *CommBuffer;
    EFI_STATUS Status = CheckPointer(v4);
    if (Status == Success) {
        if ((_BYTE *)v4 + 1) < 0xC) {
            funcs_D41[(_BYTE *)v4 + 1](v4);
        }
    }
    ...
}

```

Listing 5: A double-fetch vulnerability in Lenovo UEFI firmware

cannot be detected by RSFUZZER. As shown in Listing 4, SmuFuzz detected a cross-handler arbitrary memory write vulnerability in Dell firmware. In SMI Handler SMIHandlerA_1850, a global pointer qword_26F0 is assigned a value read from CommBuffer that can be controlled by an attacker. The memory is cleared in another SMI handler SMIHandlerB_1388. Since the value is not checked, an attacker can craft a pointer that points to SMRAM to clear any SMRAM. Listing 5 presents a double-fetch out-of-bound read vulnerability in Lenovo firmware. In SMI handler SMIHandler_CD0, attacker-controlled CommBuffer data is assigned to variable v4. The function CheckPointer checks if the pointer does not point to SMRAM. If so, it reads a byte from the pointer and checks if it is less than the length of a function pointer array funcs_D41. If it is valid, it rereads the value and uses it as an index to execute the indirect call. However, an attacker can exploit the small time window to modify the value by using a Direct Memory Access (DMA) device, causing the handler to use an out-of-bounds index for the indirect call.

5.6. Initialization Phase Fuzzing Analysis (RQ4)

An SMI handler can only be fuzzed when its module is initialized and the handler is registered. Therefore, we measure the effectiveness of initialization fuzzing by counting the successfully initialized modules and registered SMI handlers. Since RSFUZZER does not provide the module information in the output, we omit it in the final result. As shown in Table 4, SmuFuzz correctly identified and registered 49% more SMI handlers than RSFUZZER. RSFUZZER relies on the static analysis results from efiXplorer to locate the SMI handler. However, the static analysis result is not complete or accurate, resulting in a missing handler registration. For example, efiXplorer cannot correctly identify protocol consumption when the GUID data is stored in a

TABLE 4. SMM module loading and SMI handler registration information.

	RSFUZZER	SmuFuzz (without fuzzing)		SmuFuzz (with fuzzing)	
		# Handler	# Module	# Module	# Handler
Alienware 13 R3	38	19	2	64	53
Alienware X51 R3	43	43	48	62	70
ASUS P453UJ	33	35	38	44	42
ASUS UN65U	35	11	1	55	63
ASUS A407UB	43	11	1	56	49
ASUS K509FA	33	39	33	51	48
ASUS X509	23	17	5	54	53
X570 GAMING X	44	27	10	106	146
Z690 GAMING X	24	4	3	58	67
AERO 15 OLED	43	6	3	51	64
X3 Plus i7	38	11	2	43	38
X9 DT	59	35	34	46	45
HP20-c000	38	50	51	60	59
HP Obelisk 875	39	44	43	49	45
HP Z2 Mini G4	71	168	80	180	86
HP Z440	39	81	51	96	59
Obelisk PC 875-0000	51	45	52	59	66
ThinkCentre M700	50	63	65	96	79
ThinkStation P900	38	55	40	73	46
ThinkStation S30	13	0	0	0	0
Thinkpad X1 Fold	52	13	3	66	57
ThinkPad X1 Tablet	29	15	4	86	67
ThinkPad X1 2-in-1	37	55	41	62	52
Aspire 3 A315-58-52W4	21	55	48	62	62
Aspire R 13 R5-371T-595X	24	14	4	55	49
Microsoft Surface Go	30	25	12	56	49
Stealth 16 Mercedes-AMG	40	17	5	47	55
Creator Z16 HX Studio	33	1	3	45	64
Katana 15 B12VCK	37	5	3	47	51
Blade Stealth RZ09-01963	46	11	2	46	48
Blade Stealth RZ09-03102	55	47	42	51	55
	1199	1022	729	1926	1787

local variable. As such, some root handler registrations cannot be correctly identified. In contrast, SmuFuzz tries to load as many modules as possible during the initialization phase, and thus can register as many SMI handlers as possible. For SmuFuzz, 24 firmwares in the dataset can load and initialize at least 80% of the modules successfully. However, for some firmware, the successful module loading rate is low, such as 69% of HP Z2 Mini G4. We manually inspected the reasons and found that the SMM modules often follow a chain of dependencies. If a module is not correctly initialized, the modules that rely directly or indirectly on the module cannot be loaded. The initialization phase fuzzing helps load more modules and register more SMI handlers. For a comparison, without initialization phase fuzzing, SmuFuzz can only load 53% modules and register 40% SMI handlers.

5.7. Overhead and Manual Effort Analysis (RQ5)

In the composing phase, extracting the SMM modules from vendor UEFI firmware and embedding them into the infrastructure firmware takes less than five minutes for our collected firmware dataset. This process is fully automatic and requires no manual effort. The composed firmware is automatically loaded into the simulator and fuzzed in the initialization phase. This phase can take several minutes to one hour, depending on the number of modules and the time-out setting. In our experiment, we set a 30-second timeout for each module initialization function, and the initialization phase took an average of 28 minutes to complete. However, we must manually craft the DXE protocol instances to provide the dependencies for loading the SMM modules. This process involves a reverse engineering effort. However, since we synthesize the proprietary protocol with fuzz values, we only need to identify the data types of the protocol fields (i.e., integer or function pointer). This is relatively easy with the help of efiXplorer’s static analysis results. We believe

that UEFI firmware developers with access to the source code can adapt SmuFuzz to their products with negligible manual effort. The final fuzzing phase is fully automated as the SMI handler fuzzing harness of SmuFuzz is universal. The fuzzing speed can reach 60 to 100 executions per second, which is higher compared to RSFUZZER.

6. Related Work

Our approach is based on fuzzing for detecting UEFI firmware memory corruption vulnerabilities. System-level software fuzzing and UEFI memory safety have been popular topics during the last five years.

UEFI Firmware Analysis. CHIPSEC [23] is a tool for hardware and firmware debugging and manipulation, including triggering SMIs. It does not require a simulator, a harness, or source code to conduct the analysis. However, it only operates on on-device firmware. As far as we know, Yang et al [46] was the first to propose a general fuzzing framework for UEFI firmware. They run the firmware in a simulated environment and get coverage feedback from the simulator. Users are expected to target specific areas, such as specifying the fuzzing entry point and injecting the fuzz input. However, they did not mention either how to boot a closed-source vendor firmware or where to inject the fuzz input without heavy manual reverse engineering work. Tsffs [34] adopted LibAFL [17] as the fuzzer and Simics as the simulator to test UEFI firmware. Tsffs went one step further than Yang by providing a header file for users to harness the software. However, harnessing system-level software is not a straightforward process. Thus, it cannot target closed-source vendor UEFI firmware. efiXplorer [1], [2] and SPENDER [48] are static analysis tools targeting SMM executables with specific focuses. While efiXplorer only detects SMM callout vulnerability, SPENDER tracks the legacy pointer initialized by the module initialization function and unintended dereferencing in the SMI handler. STASE [40] and Excite [4] were proposed to use symbolic execution to analyze UEFI vulnerabilities by user-defined rules and attacker-controlled inputs. RSFUZZER [47] is a similar work to SmuFuzz. It focused on the problem that different SMI handlers in the same module can influence others' control flow via global variables. RSFUZZER starts from a single SMI handler, extracts knowledge from fuzzing probing, and feeds the information to multi-handler fuzzing. SmuFuzz generalizes this problem and solves it by grouping different SMI handlers based on their dependency relationships. In contrast to RSFUZZER, which only uses the Qiling [33] SMM loader, SmuFuzz offers a full UEFI runtime environment for proper SMM module initialization.

Firmware Fuzzing. Rehosting is widely adopted in firmware fuzzing. Various approaches have been proposed, including Hardware abstract layer-based methods like HALucinator [11], binary rewriting as used in Safire-fuzz [39], machine learning-assisted modeling in P2IM [14], Unicorn-based emulation in Fuzzware [35], hardware-assisted rehosting in SHiFT [29], and QEMU-based solutions such as Hoedur [36] and MultiFuzz [10]. Beyond

rehosting, existing works also tackle different aspects of firmware fuzzing, such as input space reduction (μ Emu [49] and Fuzzware [35]), input stream segmentation (HoeDur [36] and MultiFuzz [10]), interrupt-triggering (Aidfuzzer [44]), and DMA buffer inference (DICE [28]). These tools primarily target RTOS-based firmware, which interacts with a limited set of devices. In contrast, UEFI firmware accesses tens of thousands of peripheral registers, making full rehosting impractical. RTOS rehosting works by feeding fuzz inputs to all simulated device interactions, a method that does not scale to the complexity of UEFI. Therefore, decoupling the full firmware rehosting is essential for SMM application fuzzing.

System-Level Fuzzing Frameworks. kAFL [38] and LibAFL QEMU [27] are two general system-level fuzzing frameworks. kAFL utilizes the hardware features of Intel PT and Intel VT-x to provide a simulated environment and retrieve code coverage feedback. LibAFL QEMU relies on QEMU to simulate and instrument the binary. In this paper, we used the fuzzing mutation strategy Cmplog proposed by Redqueen [3] to infer constant value constraints. This technique requires accurate instruction control. Thus, we developed SmuFuzz based on LibAFL QEMU.

7. Discussion

Though SmuFuzz can be a general fuzzing framework to test any closed-source SMM applications, we discuss potential shortcomings and problems.

Fixed SMM Lock State. During the initialization phase, we use fuzzing to find a valid path that initializes the module successfully. However, once a valid path is found, we terminate fuzzing immediately and continue with the next module until no more modules can be initialized. After handling the Lock event, we start the fuzzing phase in a fixed state. The DXE protocol and other data during the initialization phase may result in different initial states for the final fuzzing phase. Since we only preserve one state during the initialization phase, some paths may not be reachable during the SMI handler fuzzing. We make a trade-off between path exploration and fuzzing availability. We leave this problem as future work.

Intact Firmware for Rehosting. Vendors may release partial firmware updates, which hinder SMM application fuzzing. Unlike full firmware rehosting, where missing parts can break the entire fuzzing, our approach partially rehosts SMM applications, enabling testing of modules that can still be initialized. For example, an incremental part of the firmware naturally lacks the necessary functional code to boot the SMM application. To solve this problem, we design and implement a fuzzing-retrofitted infrastructure. The infrastructure is a UEFI firmware that serves as the base image for booting and managing the SMM modules. The infrastructure identifies the embedded SMM modules, loads them into SMRAM, and initializes them by supplying the required data via fuzzing. Given a BIOS update, even though we cannot always obtain the intact firmware, we always receive at least intact modules. This is because

vendors only release modules as the smallest distributable units, rather than binary fragments. Therefore, SmuFuzz can embed such modules into our infrastructure and perform fuzzing on them.

Limited Support for Various Module File Types. A typical SMM module is packaged as a Position-Independent Executable (PIE) PE file. However, the UEFI standard also requires support for position-dependent or TE images. Currently, SmuFuzz can only support PIE PE files, which may lead to unsuccessful module loading. For example, SmuFuzz cannot fuzz ThinkStation S30 in the dataset since it is not a PIE file. Although PIE PE files currently represent the majority of the existing UEFI modules, we plan to add support for various module file types in the future to facilitate a comprehensive SMM module analysis.

Extra DXE Protocol Provision. Our infrastructure supports rich DXE protocols, including proprietary ones, to meet the dependencies of the SMM modules. However, when applying to new SMM applications, they may require more proprietary DXE protocols depending on the processor and platform. Although we synthesize proprietary DXE protocols with fuzz values, reverse engineering the protocol data types in the firmware binaries still requires some manual efforts. Manual effort for DXE protocol reverse engineering is minimal, as it mainly involves identifying protocol data types. Since most protocols use a single type, they can be inferred by analyzing data usage (e.g., indirect calls) with static analysis tools. Some protocol field values are fixed in the real development environment. Random fuzz input can cause unintentional crashes during the fuzzing phase, which is unavoidable. Our fuzzer can detect such a crash, but cannot prevent it from happening; thus, it causes false positives. With the help of the SmuFuzz framework, we believe that developers can easily solve this problem by accessing the source code.

Open Science and Ethical Consideration We release our source code and dataset publicly available to support open science. In our work, we adhere to the best practices and ethical standards for addressing security vulnerabilities identified during our evaluation. We reported all the vulnerabilities to the affected vendors and refrained from discussing details of unpatched vulnerabilities in this paper. We have informed all vendors when vulnerabilities in their products have been discovered and manually confirmed. The vulnerabilities were all reported to the vendors in the first week of May 2025. We tried different communication channels if we did not receive a response within one week. Before June 2025, all vendors had acknowledged our reports. While some vulnerabilities have already been patched, several vendors are still in the process of addressing the remaining vulnerabilities. We will make our best effort to collaborate with the developers to understand the vulnerabilities and mitigate them.

8. Conclusion

In this paper, we identify the key problems that hinder the deep exploration of closed-source vendor SMM

application fuzzing. To tackle these problems, we design and implement SmuFuzz. This all-in-one solution provides a fully featured UEFI runtime environment in a fuzzing-retrofitted infrastructure firmware to rehost SMM applications. We split SmuFuzz into three phases based on the SMM life cycle. SmuFuzz uses fuzzing to solve compatibility problems between the UEFI runtime environment and the SMM application, maximizing the number of loadable SMM modules. The recorded protocol and SMI handler information are used in the fuzzing phase to infer the SMI handler semantics, resulting in a deep exploration of the handlers. Our experiment shows that SmuFuzz outperforms the state-of-the-art SMM fuzzing tools regarding code exploration and vulnerability detection.

Acknowledgments

This work was supported by the European Research Council (ERC) under the consolidator grant RS³ (101045669) and by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy (EXC 2092 CASA — 390781972).

References

- [1] Alex Matrosov, Andrey Labunets, Yegor Vasilenko, Philip Lebedev. Hunting for UEFI Firmware Vulnerabilities at Scale with Automated Static Analysis. <https://i.blackhat.com/eu-20/Wednesday/eu-20-Labunets-efiXplorer-Hunting-For-UEFI-Firmware-Vulnerabilities-At-Scale-With-Automated-Static-Analysis.pdf>, 2020.
- [2] Alex Matrosov, Andrey Labunets, Yegor Vasilenko, Philip Lebedev. [Static analysis-based recovery of service function calls in UEFI firmware. https://github.com/binarly-io/Research_Publications/blob/main/EKO_2020/EKO_2020_efiXplorer.pdf, 2020.
- [3] Cornelius Aschermann, Sergej Schumilo, Tim Blazytko, Robert Gawlik, and Thorsten Holz. Redqueen: Fuzzing with input-to-state correspondence. In *Symposium on Network and Distributed System Security (NDSS)*, 2019.
- [4] Oleksandr Bazhaniuk, John Loucaides, Lee Rosenbaum, Mark R Tuttle, and Vincent Zimmer. Symbolic execution for {BIOS} security. In *USENIX Workshop on Offensive Technologies (WOOT)*, 2015.
- [5] Binarly. [BRLY-2022-003] SMM memory corruption vulnerability in SMM driver on Intel platforms. <https://www.binarily.io/advisories/brly-2022-003>, 2022.
- [6] BRLY. [BRLY-2021-019] SMM callout vulnerability in combined DXE/SMM on Fujitsu device (SMM arbitrary code execution). <https://www.binarily.io/advisories/brly-2021-019>, 2021.
- [7] BRLY. [BRLY-2022-022] SMM callout vulnerability in SMM driver (SMM arbitrary code execution). <https://www.binarily.io/advisories/brly-2022-022>, 2022.
- [8] Alexander Bulekov, Bandan Das, Stefan Hajnoczi, and Manuel Egele. No grammar, no problem: Towards fuzzing the linux kernel without system-call descriptions. In *Symposium on Network and Distributed System Security (NDSS)*, 2023.
- [9] CERT Coordination Center. Multiple race conditions due to toctou flaws in various uefi implementations. <https://kb.cert.org/vuls/id/434994>, 2024.
- [10] Michael Chesser, Surya Nepal, and Damith C Ranasinghe. Multifuzz: A multi-stream fuzzer for testing monolithic firmware. In *USENIX Security Symposium*, 2024.

- [11] Abraham A Clements, Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, Giovanni Vigna, Saurabh Bagchi, and Mathias Payer. HALucinator: Firmware re-hosting through abstraction layer emulation. In *USENIX Security Symposium*, 2020.
- [12] Tobias Cloosters, Johannes Willbold, Thorsten Holz, and Lucas Davi. SGXFuzz: Efficiently synthesizing nested structures for SGX enclave fuzzing. In *USENIX Security Symposium*, 2022.
- [13] EDK II. EDK II. <https://github.com/tianocore/edk2>, 2023.
- [14] Bo Feng, Alejandro Mera, and Long Lu. P2IM: Scalable and hardware-independent firmware testing via automatic peripheral interface modeling. In *USENIX Security Symposium*, 2020.
- [15] fiano. fiano: Go-based tools for modifying UEFI firmware. <https://github.com/linuxboot/fiano>, 2024.
- [16] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. AFL++: Combining incremental steps of fuzzing research. In *USENIX Workshop on Offensive Technologies (WOOT)*, 2020.
- [17] Andrea Fioraldi, Dominik Christian Maier, Dongjia Zhang, and Davide Balzarotti. Libafl: A framework to build modular and reusable fuzzers. In *ACM Conference on Computer and Communications Security (CCS)*, 2022.
- [18] Connor Glosner and Aravind Machiry. Fuzzuer: Enabling fuzzing of uefi interfaces on edk-2. In *Symposium on Network and Distributed System Security (NDSS)*, 2025.
- [19] Google. syzkaller, Linux syscall fuzzer. <https://github.com/google/syzkaller>.
- [20] Intel. Using host-based firmware analysis to improve platform resiliency. <https://www.intel.com/content/dam/develop/external/us/en/documents/intel-usinghbfatoimproveplatformresiliency.pdf>, 2019.
- [21] Intel. Intel® Simics® Simulator Public Release. <https://www.intel.com/content/www/us/en/developer/articles/tool/simics-simulator.html>, 2023.
- [22] Intel. Intel® 64 and IA-32 Architectures Software Developer’s Manual Combined Volumes 3A, 3B, 3C, and 3D: System Programming Guide. <https://cdrdv2.intel.com/v1/dl/getContent/671447>, 2024.
- [23] Intel. CHIPSEC: Platform Security Assessment Framework. <https://github.com/chipsec/chipsec>, 2025.
- [24] Intel Co. Gnu-efi. <https://github.com/rhboot/gnu-efi>, 2024.
- [25] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating fuzz testing. In *ACM Conference on Computer and Communications Security (CCS)*, 2018.
- [26] Lenovo. System Management Mode (SMM) BIOS Vulnerability. <https://support.lenovo.com/de/en/solutions/ps500060-system-management-mode-smm-bios-vulnerability>, 2016.
- [27] Romain Malmain, Andrea Fioraldi, and Francillon Aurélien. Libafl qemu: A library for fuzzing-oriented emulation. In *Symposium on Network and Distributed System Security (NDSS)*, 2024.
- [28] Alejandro Mera, Bo Feng, Long Lu, and Engin Kirda. Dice: Automatic emulation of dma input channels for dynamic firmware analysis. In *IEEE Symposium on Security and Privacy (S&P)*, 2021.
- [29] Alejandro Mera, Changming Liu, Ruimin Sun, Engin Kirda, and Long Lu. SHiFT: Semi-hosted Fuzz Testing for Embedded Applications. In *USENIX Security Symposium*, 2024.
- [30] Michał Zalewski. american fuzzy lop. <https://lcamtuf.coredump.cx/afl/>.
- [31] Microsoft. Smis are eeeevil (part 1). <https://learn.microsoft.com/en-us/archive/blogs/carmenr/smis-are-eeevil-part-1>, 2005.
- [32] NCC Group. A race to report a toctou: Analysis of a bug collision in intel smm. <https://www.nccgroup.com/sg/research-blog/a-race-to-report-a-toctou-analysis-of-a-bug-collision-in-intel-smm/>, 2023.
- [33] Qiling Development Team. Qiling Framework. <https://qiling.io/>, 2023.
- [34] Rowan Hart, Brandon Marken, Robert Guenzel. Intel TSFFS – Target Software Fuzzer For Simics. <https://github.com/intel/tsffs>, 2024.
- [35] Tobias Scharnowski, Nils Bars, Moritz Schloegel, Eric Gustafson, Marius Muench, Giovanni Vigna, Christopher Kruegel, Thorsten Holz, and Ali Abbasi. Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing. In *USENIX Security Symposium*, 2022.
- [36] Tobias Scharnowski, Simon Wörner, Felix Buchmann, Nils Bars, Moritz Schloegel, and Thorsten Holz. Hoedur: Embedded firmware fuzzing using multi-stream inputs. In *USENIX Security Symposium*, 2023.
- [37] Moritz Schloegel, Nils Bars, Nico Schiller, Lukas Bernhard, Tobias Scharnowski, Addison Crump, Arash Ale-Ebrahim, Nicolai Bissantz, Marius Muench, and Thorsten Holz. SoK: Prudent Evaluation Practices for Fuzzing. In *IEEE Symposium on Security and Privacy (S&P)*, 2024.
- [38] Sergej Schumilo, Cornelius Aschermann, Robert Gawlik, Sebastian Schinzel, and Thorsten Holz. kafl: hardware-assisted feedback fuzzing for os kernels. In *USENIX Security Symposium*, 2017.
- [39] Lukas Seidel, Dominik Maier, and Marius Muench. Forming Faster Firmware Fuzzers. In *USENIX Security Symposium*, 2023.
- [40] Md Shafiuazzaman, Achintya Desai, Laboni Sarker, and Tevfik Bultan. Stase: Static analysis guided symbolic execution for uefi vulnerability signature generation. In *ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2024.
- [41] tianocore. Open virtual machine firmware. <https://github.com/tianocore/tianocore.github.io/wiki/OVMF>, 2017.
- [42] UEFI Forum, Inc. Unified Extensible Firmware Interface (UEFI) Specification. https://uefi.org/sites/default/files/resources/UEFI_Spec_2_10_A_Aug8.pdf, 2024.
- [43] UEFITool. UEFITool: UEFI firmware image viewer and editor. <https://github.com/LongSoft/UEFITool>, 2024.
- [44] Wang, Jianqiang and Wang, Qinying and Scharnowski, Tobias and Shi, Li and Woerner, Simon and Holz, Thorsten. Aidfuzzer: Adaptive interrupt-driven firmware fuzzing via run-time state recognition. In *USENIX Security Symposium*, 2025.
- [45] Wikipedia. Universally unique identifier. https://en.wikipedia.org/wiki/Universally_unique_identifier, 2025.
- [46] Zhenkun Yang, Yuriy Viktorov, Jin Yang, Jiewen Yao, and Vincent Zimmer. Uefi firmware fuzzing with simics virtual platform. In *Design Automation Conference (DAC)*, 2020.
- [47] Jiawei Yin, Menghao Li, Yuekang Li, Yong Yu, Boru Lin, Yanyan Zou, Yang Liu, Wei Huo, and Jingling Xue. Rsfuzzer: Discovering deep smi handler vulnerabilities in uefi firmware with hybrid fuzzing. In *IEEE Symposium on Security and Privacy (S&P)*, 2023.
- [48] Jiawei Yin, Menghao Li, Wei Wu, Dandan Sun, Jianhua Zhou, Wei Huo, and Jingling Xue. Finding smm privilege-escalation vulnerabilities in uefi firmware with protocol-centric static analysis. In *IEEE Symposium on Security and Privacy (S&P)*, 2022.
- [49] Wei Zhou, Le Guan, Peng Liu, and Yuqing Zhang. Automatic firmware emulation through invalidity-guided knowledge inference. In *USENIX Security Symposium*, 2021.

Appendix A. Extended Figure

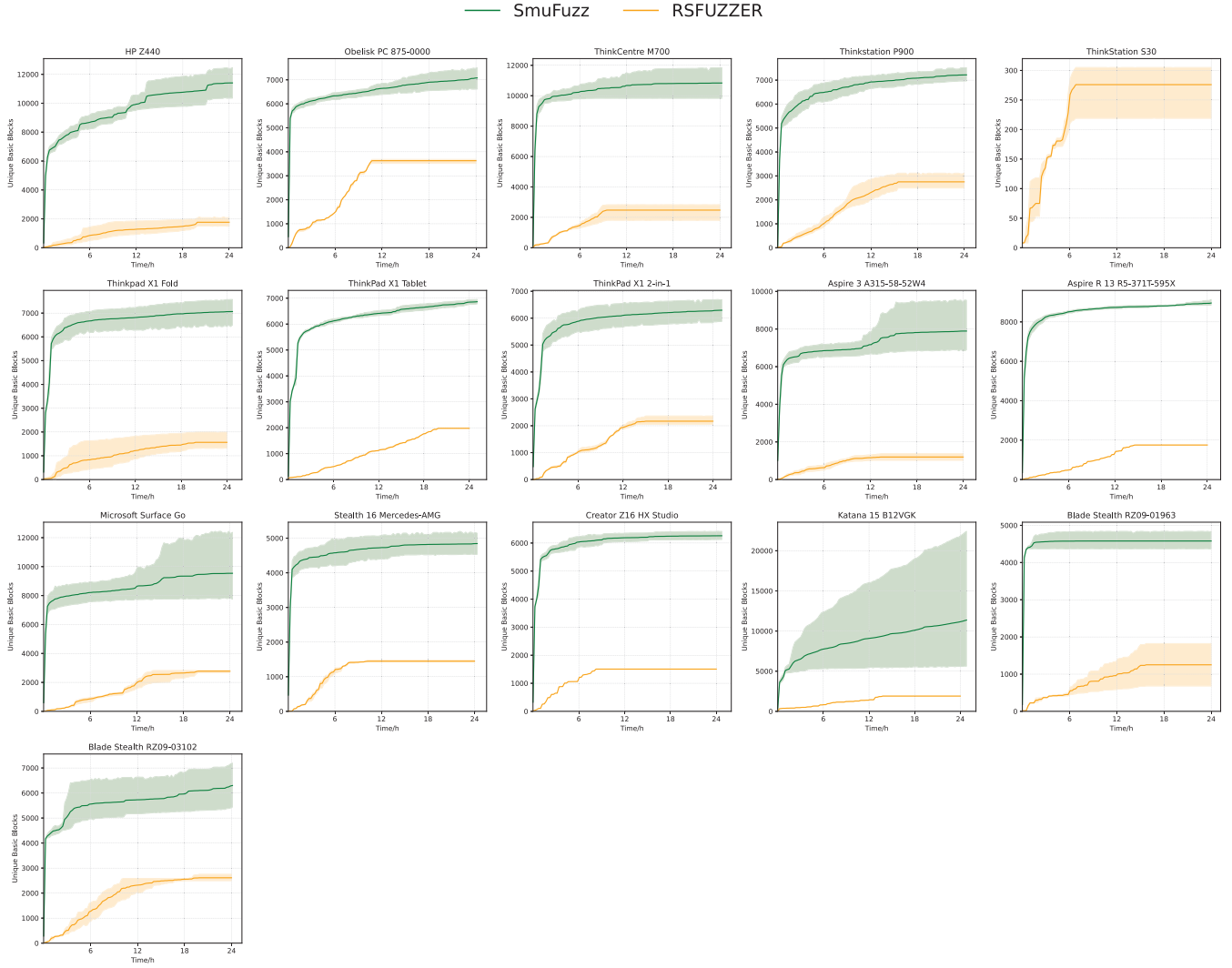


Figure 7. Basic block coverage achieved by RSFUZZER and SmuFuzz over 24 hours for five times. We plot the median and 95% confidence interval.

Appendix B. Meta-Review

The following meta-review was prepared by the program committee for the 2026 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

B.1. Summary

This paper presents SmuFuzz, a fuzzing framework specifically designed for closed-source SMM applications. SmuFuzz provides a custom infrastructure that partially rehosts SMM applications within a complete UEFI runtime, enabling proper initialization and execution. SmuFuzz outperforms existing fuzzers and discovers 38 previously unknown vulnerabilities.

B.2. Scientific Contributions

- Creates a New Tool to Enable Future Science
- Provides a Valuable Step Forward in an Established Field

B.3. Reasons for Acceptance

- 1) This paper provides a valuable step forward in an established field, with a new approach that allows firmware rehosting and fuzzing. This approach enables SMM applications to be rehosted by providing the necessary protocol support based on the observation that these applications actually depend on protocols instead of hardware interactions.
- 2) This paper creates a new tool to enable future science, with significant engineering contributions. The authors implement SmuFuzz, which requires roughly 10K lines of code. The authors will make it publicly available to support future research.
- 3) This paper works on an important problem, vulnerability identification of UEFI firmware, that could compromise the OS while remaining undetected. It makes impactful findings as 38 new vulnerabilities are revealed, affecting major vendors.